King Saud University
College of Computer and Information Sciences
Department of Information Technology

IT362: Principles of Data Science

Semester-2, 1447H

Phase 3: Modelling and Communication

**Logbook**

Topic: Sentiment analysis of reviews of beauty centres

Prepared by

| Section No | Name | ID |
|---|---|---|
| **73059** | Leen Almuqhim | 445201272 |
| | Remas Alayiedh | 445202140 |
| | Lubna Alqifari | 445202104 |
| | Aljoury Alreemi | 445200074 |
| | Khadijah Alshehri | 445202138 |

# I. Part 1: Modelling

## Step 1: Modelling Dataset Preparation

### 1. Loading Data

In this phase, the cleaned dataset obtained from Phase 2 was prepared for machine learning modelling. The focus was on transforming textual data into a format suitable for classification models.

```
df = pd.read_csv('/content/primary_google_beauty_reviews_cleaned (1).csv')
df.head()
df.info()
```

The dataset contains the processed text reviews and sentiment labels required for modelling.

### 2. Selecting Features & Target

The relevant columns were selected for modelling. The 'processed_text' column was used as the input feature, and 'sentiment_label' was used as the target variable. Missing values were removed to ensure data quality.

```
text_col = "processed_text"
target_col = "sentiment_label"

df = df[[text_col, target_col]].dropna()
X_text = df[text_col].astype(str)
y = df[target_col]
```

After cleaning, the dataset contained only valid records with no missing values.

### 3. Bag of word

The Bag of Words technique was applied to convert text into numerical features based on word frequency.

```
bow_vectorizer = CountVectorizer()
X_bow = bow_vectorizer.fit_transform(X_text)
```

Each review is represented as a vector of word counts.

## 4. TF-IDF

TF-IDF was applied to assign importance weights to words, reducing the impact of common terms.

```
tfidf_vectorizer = TfidfVectorizer()
X_tfidf = tfidf_vectorizer.fit_transform(X_text)
```

TF-IDF provides a more informative representation compared to Bag of Words.

## 5. Train-Test split

The dataset was split into training and testing sets using an 80/20 ratio. Stratified sampling was used to preserve class distribution.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

The data was successfully divided into training and testing sets while maintaining class balance.

**Challenges:** A small number of missing values were found in the processed text column. This issue was resolved by removing incomplete records to ensure smooth processing during vectorization.

Another challenge was ensuring that the class distribution remained balanced after splitting the dataset. This was handled using stratified sampling to preserve the original distribution of sentiment classes.

**Outcome:** The dataset was successfully prepared for modelling. Text data was converted into numerical representations using Bag of Words and TF-IDF, and the dataset was split into training and testing sets. All validation checks confirmed that the data is clean, consistent, and ready for building and evaluating machine learning models.

## Step 2: Baseline Model Implementation

**What was done:** A baseline classification model was developed using Logistic Regression to predict sentiment labels (positive, neutral, negative) from customer reviews.

**Why it was necessary**: The baseline model serves as a benchmark to evaluate whether more advanced models provide meaningful improvements in performance. It establishes a minimum expected performance level for the classification task.

**How it was implemented**: The processed review text was transformed into numerical features using TF-IDF vectorization. The dataset was split into training and testing sets using stratified sampling. A Logistic Regression model was trained on the training data and evaluated on the test set.

**Code Snippet:**

```
baseline_model = LogisticRegression(max_iter=1000)
baseline_model.fit(X_train, y_train)
y_pred_baseline = baseline_model.predict(X_test)
```

**Evaluation Metrics:**

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1-score (weighted)
- Confusion Matrix

**Results:**

| | | | |
|---|---|---|---|
| Accuracy: 0.893 | Precision: 0.881 | Recall: 0.893 | F1-score: 0.873 |

**Observations:**

- The model performs very well overall, especially for the positive class, which achieves high recall and F1-score.
- The negative class shows moderate performance, indicating some misclassification.
- The neutral class is not well predicted, likely due to class imbalance and limited training examples.

**Challenges Encountered:**

- Class imbalance in the dataset, particularly the small number of neutral reviews, affected model performance.
- Multi-class evaluation required careful selection of metrics (weighted averaging).

**How Challenges Were Addressed:**

- Used weighted evaluation metrics to account for class imbalance.
- Maintained consistent preprocessing and feature extraction to ensure fair comparison with other models.

## Step 3: Advanced Model 1 – Support Vector Machine (SVM)

**What was done:** A Support Vector Machine (SVM) classification model was developed to predict sentiment labels (positive, neutral, negative) from customer reviews.

**Why it was necessary:** The SVM model was implemented as an advanced model to improve classification performance compared to the baseline model. It is particularly effective for text classification tasks due to its ability to handle high-dimensional feature spaces such as TF-IDF representations.

**How it was implemented:** The model was trained using the same TF-IDF feature representation and the same training and testing datasets used in previous models to ensure fair comparison. A Linear Support Vector Classifier (LinearSVC) was used with a fixed random state. After training, predictions were generated on the test set and evaluated using multiple performance metrics.

**Code Snippet:**

```
svm_model = LinearSVC(random_state=42)
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
svm_accuracy = accuracy_score(y_test, y_pred_svm)
print(classification_report(y_test, y_pred_svm))
```

**Evaluation Metrics:**

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1-score (weighted)
- Confusion Matrix

**Results:**

| | | | |
|---|---|---|---|
| Accuracy: 0.943 | Precision: 0.943 | Recall: 0.943 | F1-score: 0.935 |

**Observations:**

• The model achieved high overall accuracy and strong performance on positive and negative classes.
• The positive class showed excellent recall and precision.
• The negative class showed good performance with minor misclassification.
• The neutral class was poorly predicted due to class imbalance and limited samples.

**Challenges Encountered:**

• Class imbalance affected the performance of the neutral class.

• Some classes had significantly fewer samples, making it difficult for the model to learn effectively.

**How Challenges Were Addressed:**

• Used weighted evaluation metrics to ensure fair performance measurement across classes.

• Maintained consistent preprocessing and feature extraction (TF-IDF) for fair comparison.

## Step 4: Advanced Model 2 – Random Forest Implementation

**What was done:**

An advanced classification model was developed using the Random Forest algorithm to predict sentiment labels (positive, neutral, negative) from customer reviews.

**Why it was necessary:**

The Random Forest model was implemented to improve classification performance compared to the baseline model. As an ensemble learning method, it combines multiple decision trees to produce more accurate and robust predictions.

**How it was implemented:**

The model was trained using the same TF-IDF feature representation and the same training and testing datasets used in previous models to ensure fair comparison. A Random Forest classifier with 100 decision trees (n_estimators=100) was used. After training, predictions were generated on the test set and evaluated using multiple performance metrics.

**Code Snippet:**

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
```

**Evaluation Metrics:**

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1-score (weighted)
- Confusion Matrix

**Results:**

| | | | |
|---|---|---|---|
| Accuracy: 0.919 | Precision: 0.901 | Recall: 0.919 | F1-score: 0.904 |

**Observations:**

• The model achieved high overall performance, with strong accuracy and F1-score.

• The positive class showed excellent performance with very high recall.

• The negative class performance was moderate, indicating some misclassification.

• The neutral class was poorly predicted due to class imbalance.

**Challenges Encountered:**

• Class imbalance significantly affected the model's ability to correctly predict the neutral class.

• Some classes had very few samples, making it difficult for the model to learn meaningful patterns.

**How Challenges Were Addressed:**

• Used weighted evaluation metrics to better reflect performance across imbalanced classes.

• Maintained consistent preprocessing and feature extraction (TF-IDF) to ensure fair comparison across models.

## Step 5: Model Evaluation & Selection

**What was done:**

All models were evaluated and compared to determine the best-performing model.

**Why it was necessary:**

To identify the most accurate and reliable model for sentiment prediction based on performance metrics.

**How it was implemented:**

The results of all models were compared using accuracy, precision, recall, and F1-score. Visualizations such as accuracy comparison charts and F1-score comparison charts were also created.

**Results Comparison:**

• SVM achieved the highest performance across all metrics.

• Logistic Regression and Random Forest showed similar but lower performance.

**Observations:**

• SVM consistently outperformed other models in both accuracy and F1-score.

• The neutral class remained the weakest across all models.

**Best Model:**

SVM (LinearSVC)

**Reason for Selection:**

SVM achieved the highest F1-score and accuracy, making it the most reliable model for classification. It demonstrated strong generalization and consistent performance across different evaluation metrics.

**Outcome:**

The SVM model was selected as the final model for prediction due to its superior performance.

## Step 6: Applying Model to New Reviews

**What was done:** After selecting the best-performing model (SVM), the model was applied to new, unseen customer reviews to evaluate its real-world applicability.

**Why it was necessary:** This step was performed to demonstrate that the model is not limited to evaluation on test data and can generalize to new inputs. It validates the practical usability of the model for real-world sentiment analysis tasks.

**How it was implemented:** New review samples were manually defined and passed through the same preprocessing and TF-IDF transformation pipeline used during training. The trained SVM model was then used to predict sentiment labels.

**Code Snippet:**

```
new_reviews_processed = [preprocess_new_review(text) for text in
new_reviews]
new_reviews_tfidf = tfidf_vectorizer.transform(new_reviews_processed)
new_predictions = svm_model.predict(new_reviews_tfidf)
```

**Observations:**

- The model correctly classified clearly positive and negative reviews.
- All sample reviews were predicted as positive in some initial attempts.
- The neutral class was not predicted, even for moderately worded reviews.

**Challenges Encountered:**

- The model showed a strong bias toward the positive class.
- Neutral sentiment was difficult to predict due to very limited training samples.

**How Challenges Were Addressed:**

- More explicit review examples were tested to better evaluate model behavior.
- Results were interpreted in the context of class imbalance observed during evaluation.

**Conclusion:** The model demonstrates good performance for clear sentiment categories but has limitations in handling neutral sentiment. This highlights the importance of balanced datasets for multi-class classification tasks.